UNIT 1

Introduction To Database Management System

Basic Definitions

- **Data**: Known facts that can be recorded and have an implicit meaning.
- **Database**: Systematic collection of related data.
- **Database Management System (DBMS)**: A collection of software to facilitate the creation and maintenance of a DB.
- **Database System**: The DBMS software together with the data. Sometimes, applications are also included.

Database Example

- **College Database**
  - ✓ *Student file*
  - ✓ *Course file*
  - ✓ *Section file*
  - ✓ *Grade-Report file*
  - ✓ *Pre-requisite file etc.*

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

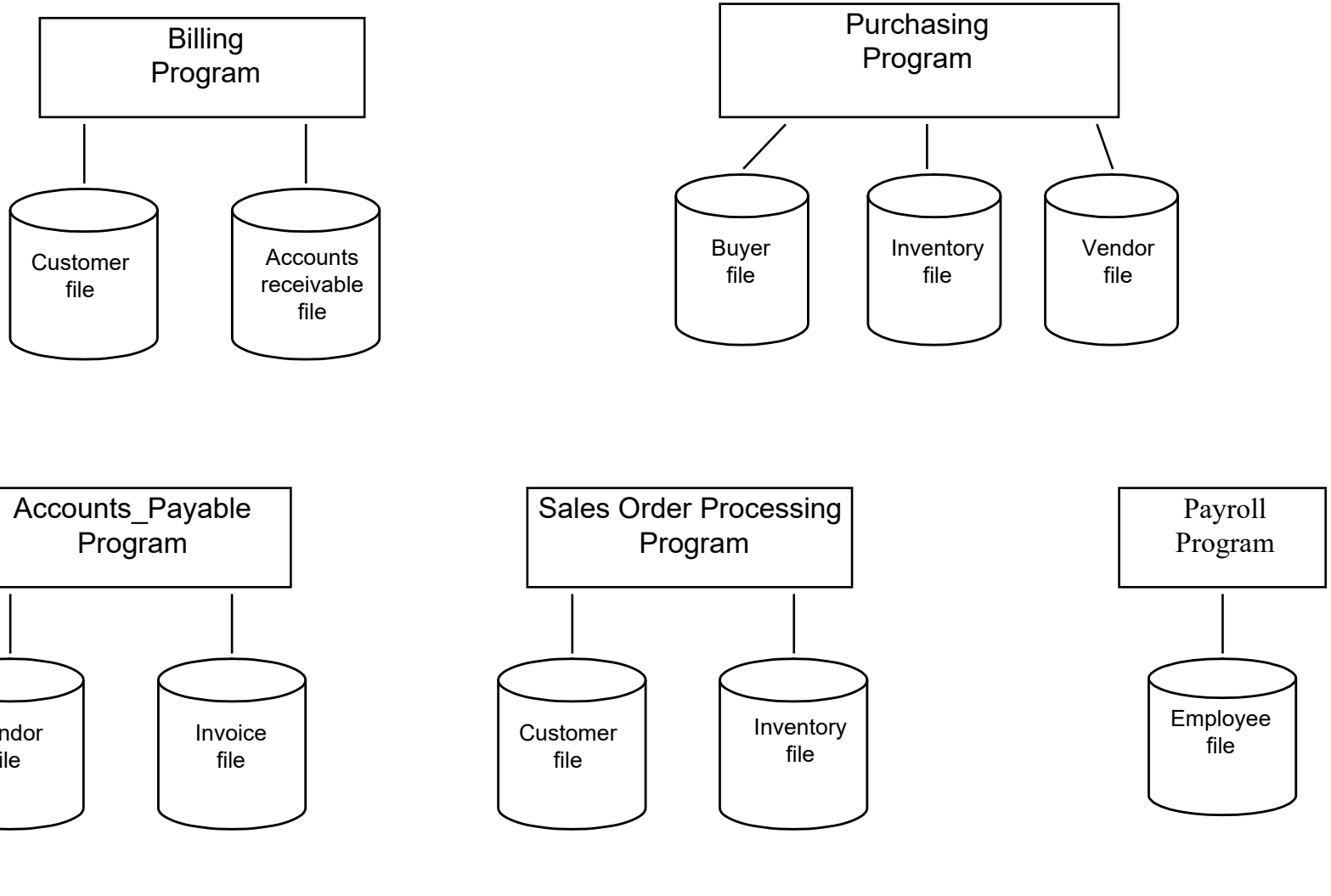
Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

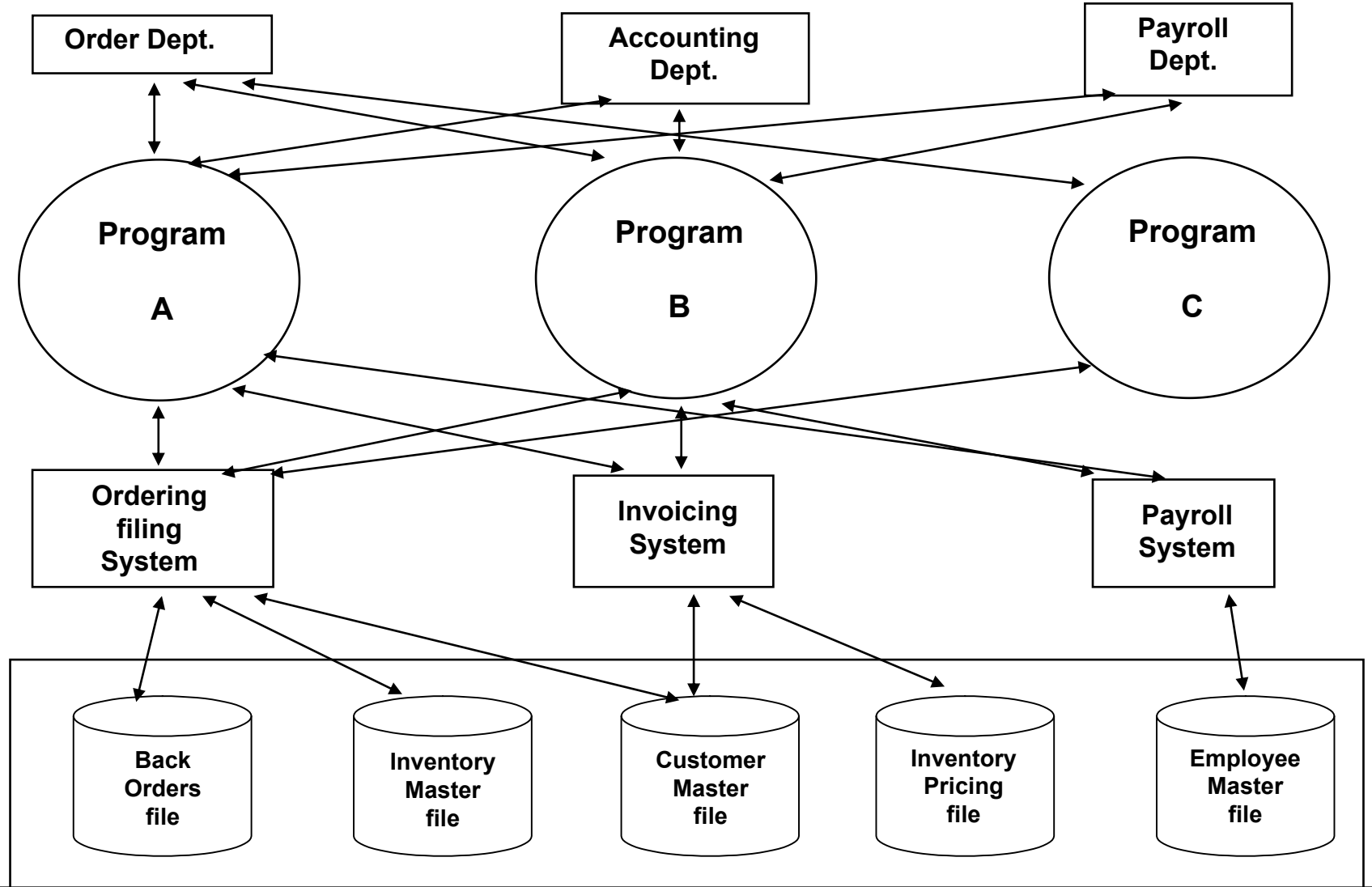
Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Difference between Traditional File Processing Vs. DBMS

File Processing Systems



Database Approach



Characteristics of Database

- Self-describing nature
- Insulation between program, data and data abstraction
- Support for multiple views of data
- Sharing of data and Multiuser Transaction processing

Actors on the Scene

(job involves day-to-day use of d/b)

- Database Administrator
- Database Designer
- End User
- System Analysts and Application Programmers

Workers Behind the Scene

(People responsible for development & operation of DBMS)

- DBMS system designers & implementers
- Tool Developers
  - ✓ For performance monitoring (SQL Server Profiler)
  - ✓ Natural language or graphical interface (SQL Lite D/b browser)
- Operators & maintenance

Advantages of DBMS

- ✓ Controlling Redundancy
- ✓ Restricting unauthorized access
- ✓ Providing storage structures for efficient query processing
- ✓ Query optimization
- ✓ Providing backup & recovery
- ✓ Representing complex relations and data
- ✓ Enforcing integrity constraints

Advantages (Continued..)

- ✓ Permitting Inferencing and actions using rules
- ✓ Additional advantages
  - Potential for enforcing standards
  - Reduced application development time
  - Flexibility
  - Availability of up-to-date information
  - Economies of scale

DBMS Vs. File System

Difference Factor	File System	DBMS
Definition	Is an abstraction to store, retrieve, manage and update a set of files	A collection of interrelated data and a set of programs to access it
Data Redundancy	Each user defines and implements needed file for a specific application	A single repository of data is maintained that is defined once and accessed by many users
Sharing of data	Doesn't allow sharing of data or data sharing is very complex	Data can be shared easily due to centralized storage
Data consistency	When data is redundant it is difficult to update	Less data redundancy so data remains consistent
Difficult to search/access data	It becomes difficult as need to refer multiple files	Very easy and user friendly. Query operations are already available

DBMS Vs. File System

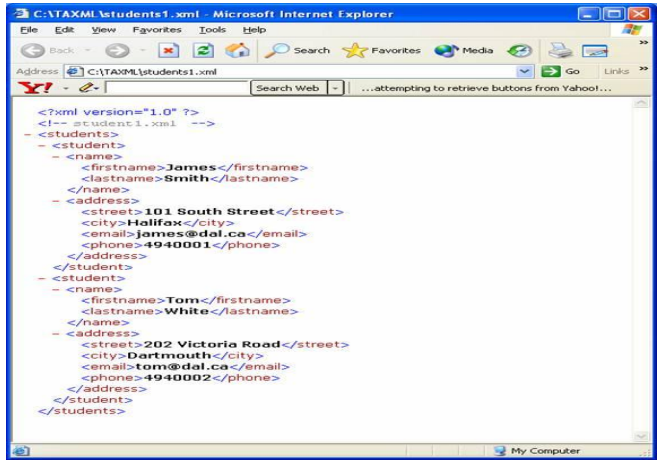
Difference Factor	File System	DBMS
Data Isolation	No standard format of data, data is scattered in various files	Due to centralized system, format of similar type of data remains same
Data Integrity	Data in d/b must follow or satisfy some rules or constraints to avoid invalid data entry	Maintains data integrity by providing constraints
Security problems	No or less security. Majorly provided are locks, guards	High level of security as encryption, passwords
Concurrent access anomalies	File system doesn't provide any remedy for it	Provides functionality for concurrent access of transactions

A Brief History D/B Applications

- ✓ Early D/b applications using hierarchical & network systems
- ✓ Providing application flexibility with relational databases
- ✓ Object oriented applications & the need for more complex databases
- ✓ Interchanging data on the web for E-commerce
- ✓ Extending database capabilities for new applications

Hierarchical System

- ✓ Uses one-to-many relationship
- ✓ Presents data in tree-like structure
- ✓ E.g. IBM's IMS(Information management system), .xml file




```
01 LIBRARY-SEGMENT.
      05 BOOK-ID PIC X(5).
      05 ISSUE-DATE PIC X(10).
      05 RETURN-DATE PIC X(10).
      05 STUDENT-ID PIC A(25).
01 BOOK-SEGMENT.
      05 BOOK-ID PIC X(5).
      05 BOOK-NAME PIC A(30).
      05 AUTHOR PIC A(25).
01 STUDENT-SEGMENT.
      05 STUDENT-ID PIC X(5).
      05 STUDENT-NAME PIC A(25).
      05 DIVISION PIC X(10).
```

Disadvantages (When not to use DBMS)

- ✓ Huge investment in s/w, h/w & training
- ✓ Overhead for providing security, concurrency control, recovery, and integrity functions
- ✓ Simple, well defined, not expected to change applications
- ✓ Multiple user access to data is not required

Database System Applications

Databases are widely used. Here are some representative applications:

- Banking

  For customer information, accounts, and loans, and banking transactions.

- Airlines:

  For reservations and schedule information.

- Universities:

For student information, course registrations, and grades.

Database System Applications

- Credit card transactions

  For purchases on credit cards and generation of monthly statements.

- Telecommunication:

  For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.

- Finance

  For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds.

Database System Applications

- Sales:

  For customer, product, and purchase information.

- Manufacturing:

  For management of supply chain and for tracking production of items in factories, inventories of items in warehouses/stores, and orders for items.

- Human resources:

  For information about employees, salaries, payroll taxes and benefits, and for generation of paychecks.

View of Data

- A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data.
- A major purpose of a database system is to **provide users with an abstract view of the data.**
- That is, the system **hides certain details** of how the data are stored and maintained.

Data Abstraction

- The system hides certain details of how the data are stored and maintained.

Data Model

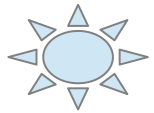
- Data Model a Collection of concepts that can be used to describe the structure of database – provides the necessary means to achieve **data abstraction**

Why Data Model?

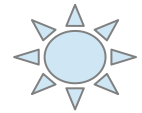
- Simple graphical representation of user requirement
- Improves interaction among designer, the application programmer and end user
- Data model organizes data from different perspectives
- Helps to make a clear picture of user requirement

- External Level: (View Level)
  - User's view of database
  - Part of database is displayed as per user requirement
  - Provides security mechanism by hiding parts of the DB from certain users
  - Permits customized access to data
  - Any data available to user must be derived from the conceptual level
  - E.g.
    - sales data, profit data etc. visible to special users of company
    - Facebook data

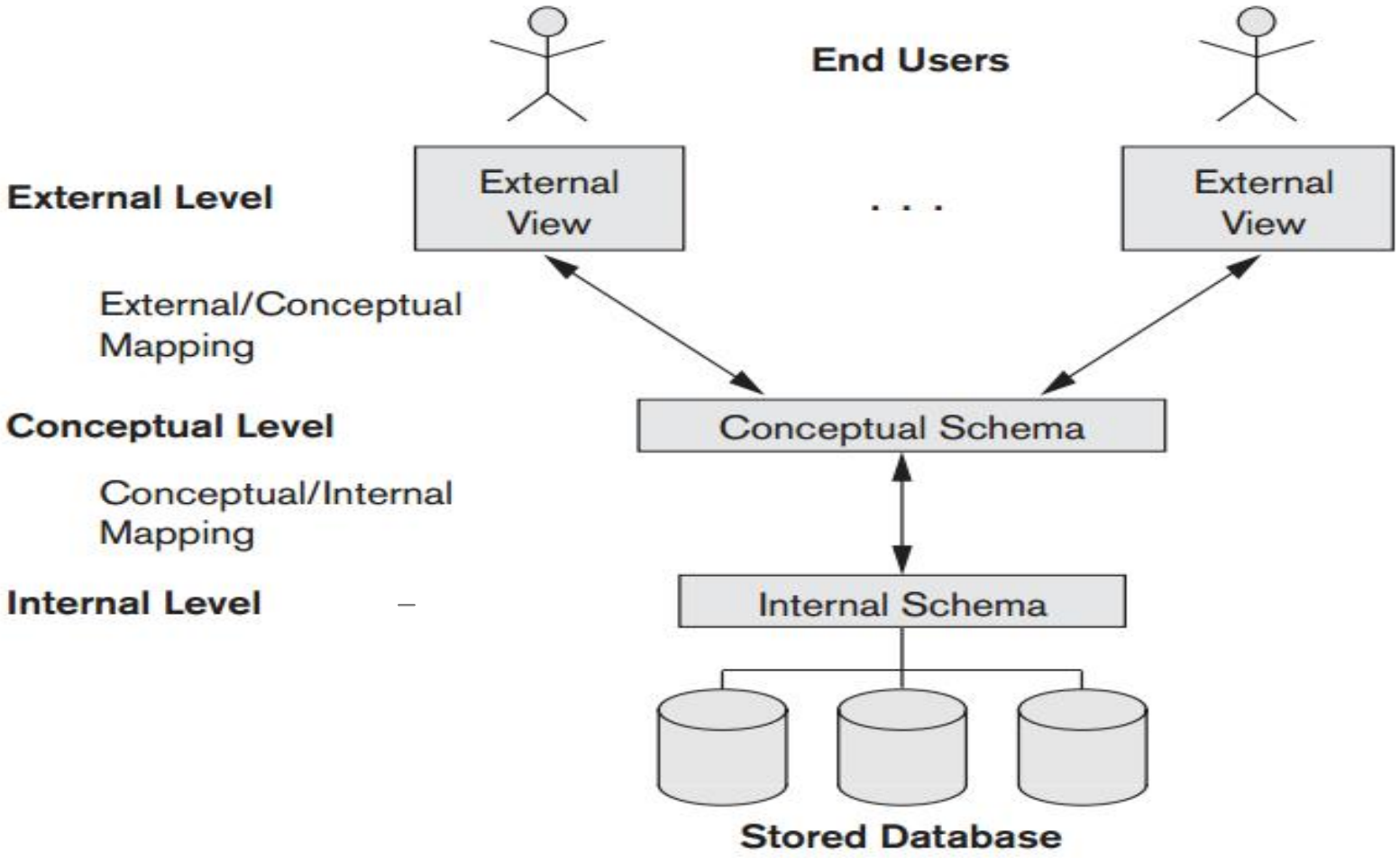
- Logical View- Conceptual Level :
  - Logical structure of database (DBA view)
  - What data is stored
  - Relationships among data
  - Independent of storage consideration
  - Representational/implementation data model
  - Represents
    - Entities, attributes, relations
    - Constraints
    - Semantic information on data
    - Security, integrity information



- Physical View-Internal Level :
  - How data is stored in the database
  - Physical representation of the DB on the computer to achieve optimal run-time performance and storage space utilization.
  - It considers:
    - Storage space allocation for data and indexes
    - Record description for storage
    - Record placement
    - Data compression, encryption



Three Schema Architecture and Data Independence



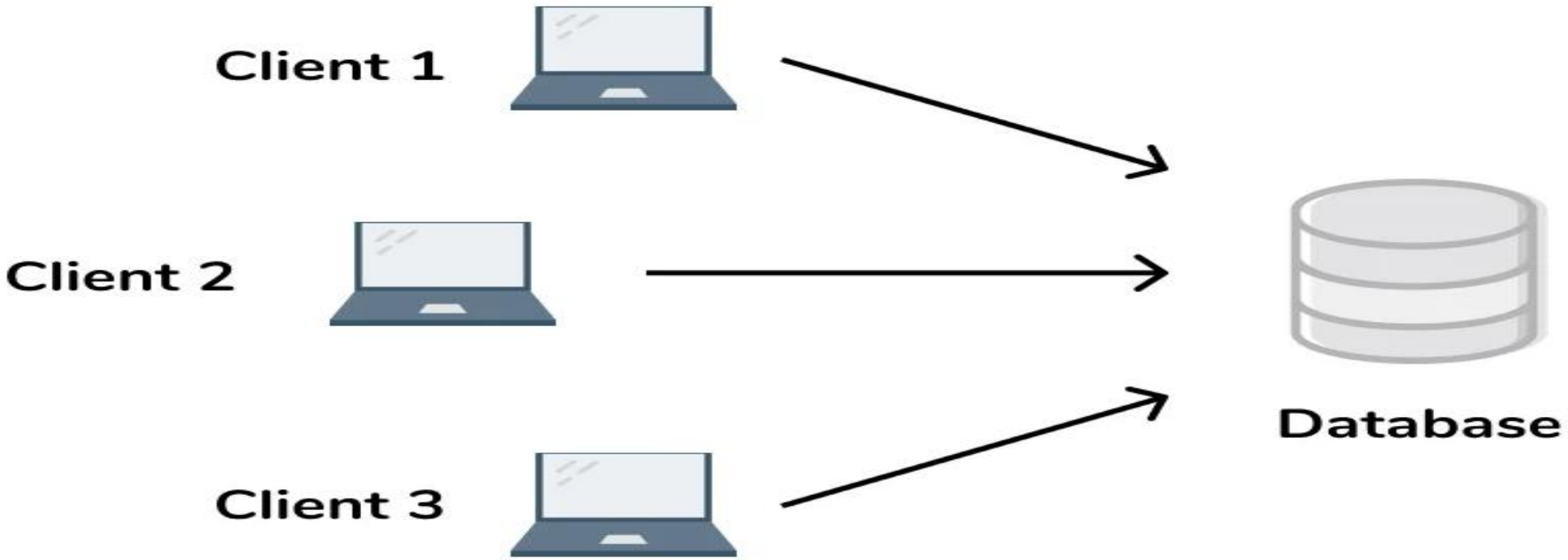
Two-tier Architecture

- This is where you have direct communication between a client and a server with no intermediary. It is divided into two parts;

- 1.) Client Application.

- 2.) Database.

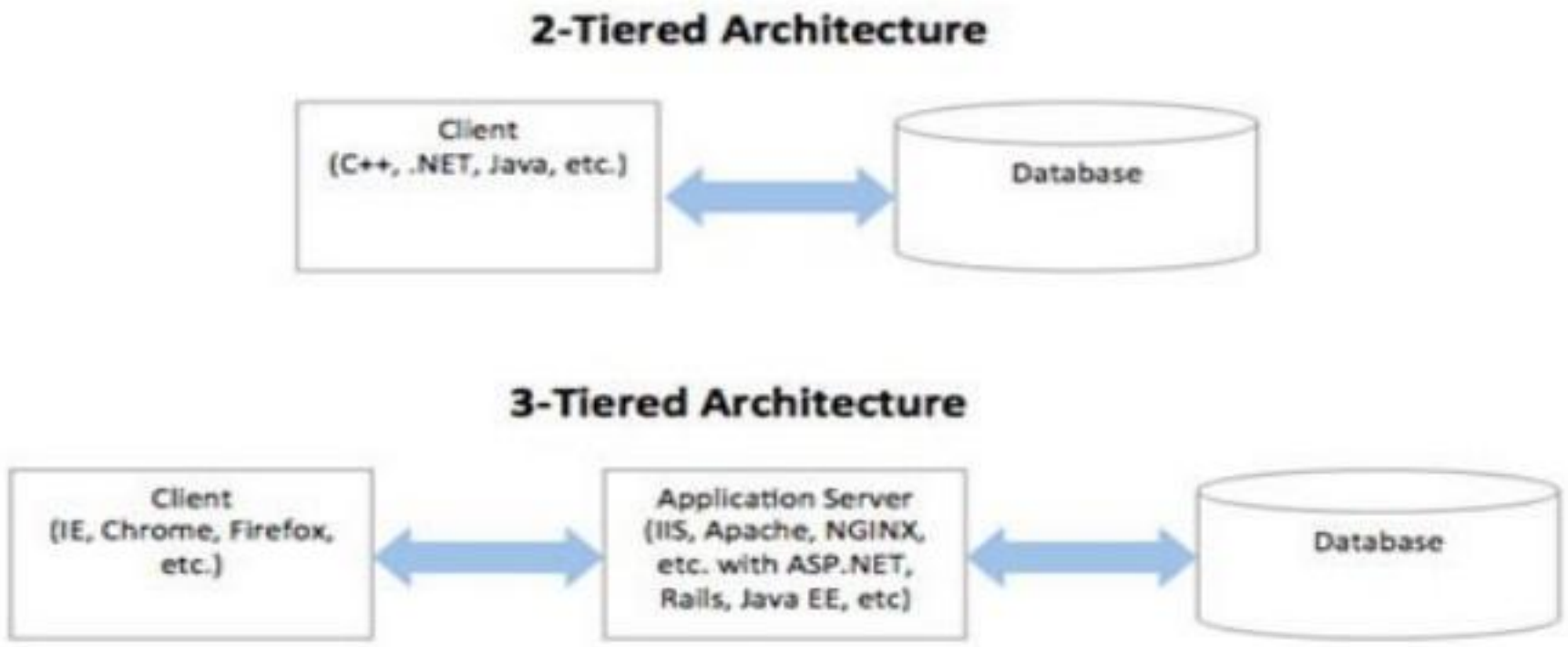
Two Tier Architecture



Two-tier Architecture

- The client in a Two-tier architecture application has the code written for saving data in the database.
- The client sends a request to the server, where it then processes the request and sends back the data.
- Meaning the client handles both the Presentation layer (application interface) and Application layer (logical operations),
- while the server system handles the database layer.

Three-tier Architecture



Three-tier Architecture

- The Three-tier Architecture is divided into 3 parts;

1.) Presentation Layer (Client tier)
2.) Application Layer (Business tier)
3.) Database Layer (Data tier)

Presentation Layer

- The Presentation Layer is the **topmost** layer of an application.
- This is the layer seen when using the software(**Interface**, web pages).
- By using the software we access web pages.
- Its main function is to communicate with the application layer.
- This layer passes information which is given by the user in terms of keyboard actions and mouse clicks to the application layer.

Application Layer

- The Application Layer is also known as the **Business logic layer.**
- Here's where we find logic controls and functionality that processes data received from the presentation layer and database layer.
- It acts as an **intermediary** between the presentation and database layer.

Database Layer

- The Database Layer is the layer that stores data with the retrieval storage and execution methods made by the application layer.
- It contains methods that connect to the database and performs the required actions needed.
- These are Insert, update or delete. Just to name a few.

Three-tier Architecture benefits.

- Three-tier Architecture provides the following benefits.
- Scalability — **Each tier can scale horizontally.** For example, you can load-balance the Presentation tier among three servers to satisfy more Web requests without adding servers to the Application and Data tiers.
- Performance — Because the Presentation tier can cache requests, network utilization is minimized, and the load is reduced on the Application and Data tiers. If needed, you can load-balance any tier.
- Availability — If the Application tier server is down and caching is sufficient, the Presentation tier can process Web requests using the cache.

What are database languages?

- Database languages, also known as query languages or data query languages

Database Languages

- A database system provides
- a data-definition language to **specify** the database schema
- a data-manipulation language to express database **queries** and updates.

Data definition language (DDL)

- Data definition language (DDL) creates the framework of the database by specifying the database schema, which is the structure that represents the organization of data.

- Its common uses include the creation and alteration of tables, files, indexes and columns within the database.

- This language also allows users to rename or drop the existing database or its components. Here's a list of DDL statements:

Data definition language (DDL)

- CREATE: Creates a new database or object, such as a table, index or column
- ALTER: Changes the structure of the database or object
- DROP: Deletes the database or existing objects
- RENAME: Renames the database or existing objects

Data manipulation language (DML)

- Data manipulation language (DML) provides operations that handle user requests, offering a way to access and manipulate the data that users store within a database.

- Its common functions include inserting, updating and retrieving data from the database. Here's a list of DML statements:

Data manipulation language (DML)

- INSERT: Adds new data to the existing database table
- UPDATE: Changes or updates values in the table
- DELETE: Removes records or rows from the table
- SELECT: Retrieves data from the table or multiple tables

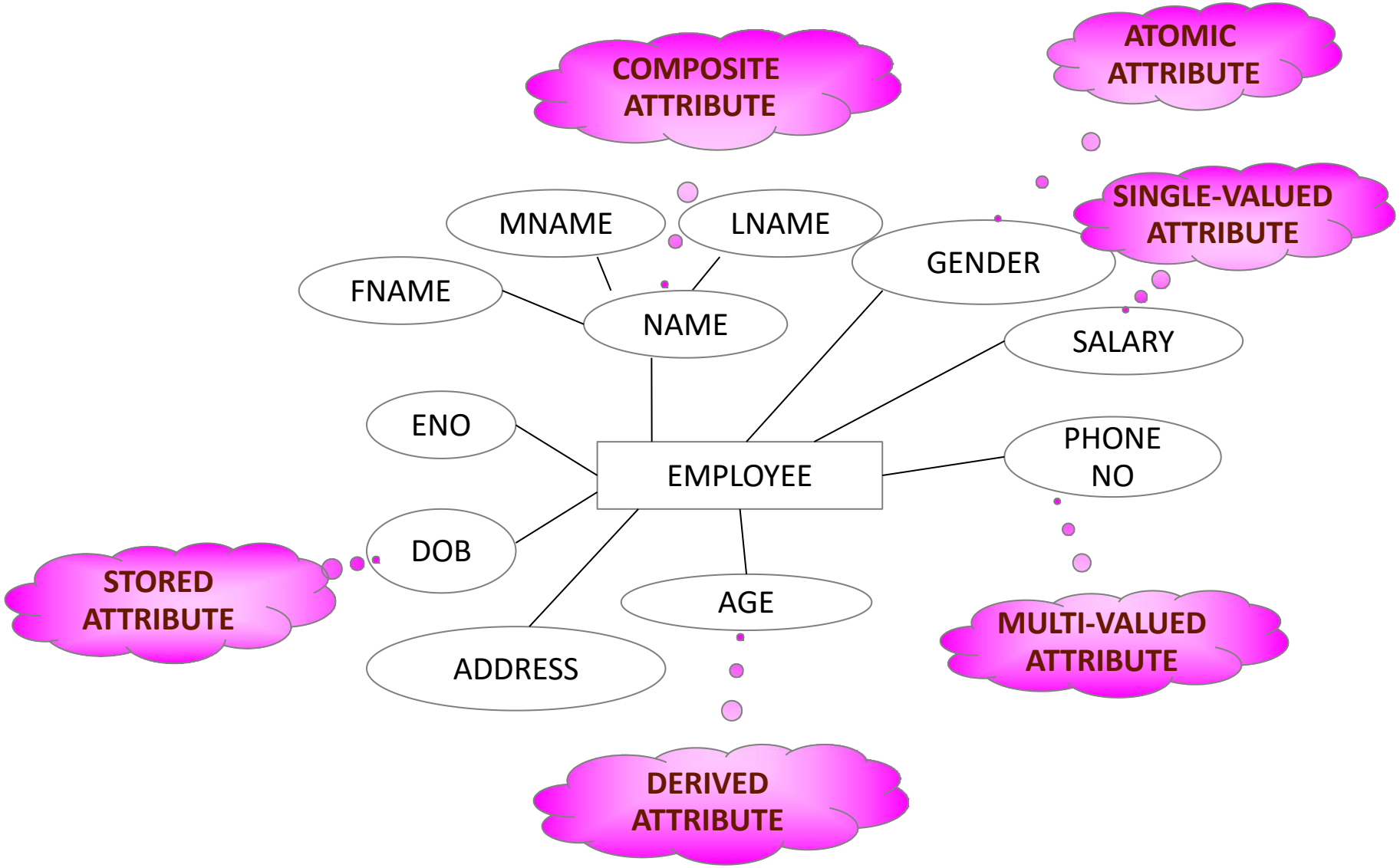
E-R Model

- High level conceptual data model
- No implementation details
- Used to communicate to non technical users
- Describes the data as entities,relationships and attributes.

Basic Concepts

- Entity:
  - Thing in the real world with an independent existence.
  - e.g. A particular person, a car

    A Company, a job, a university course
- Attributes:
  - Particular properties that describe an entity


COMPOSITE ATTRIBUTE
ATOMIC ATTRIBUTE
SINGLE-VALUED ATTRIBUTE
MNAME
LNAME
FNAME
NAME
GENDER
SALARY
ENO
EMPLOYEE
PHONE NO
DOB
STORED ATTRIBUTE
AGE
ADDRESS
MULTI-VALUED ATTRIBUTE
DERIVED ATTRIBUTE

Composite Vs Atomic/Simple Attributes

Atomic Attributes : Attributes that are not divisible.

- ✓ Value of composite attribute is concatenation of the values of constituent atomic attributes

Composite Attributes : Can be divided into smaller sub parts which represent more basic attributes with independent meanings

Single Valued Vs Multi-Valued Attributes

Single Valued Attributes : Attributes having single value for a particular entity. E.g. salary, DOB etc.

Multi-Valued Attributes : Attributes having a set of values for a particular entity. E.g. phone number, color of a car

Stored Vs Derived Attributes

- ✓ Two or more attribute values are related.
- ✓ **E.g.** age and DOB of a person.
- ✓ For a person age of a person can be derived from DOB of a person

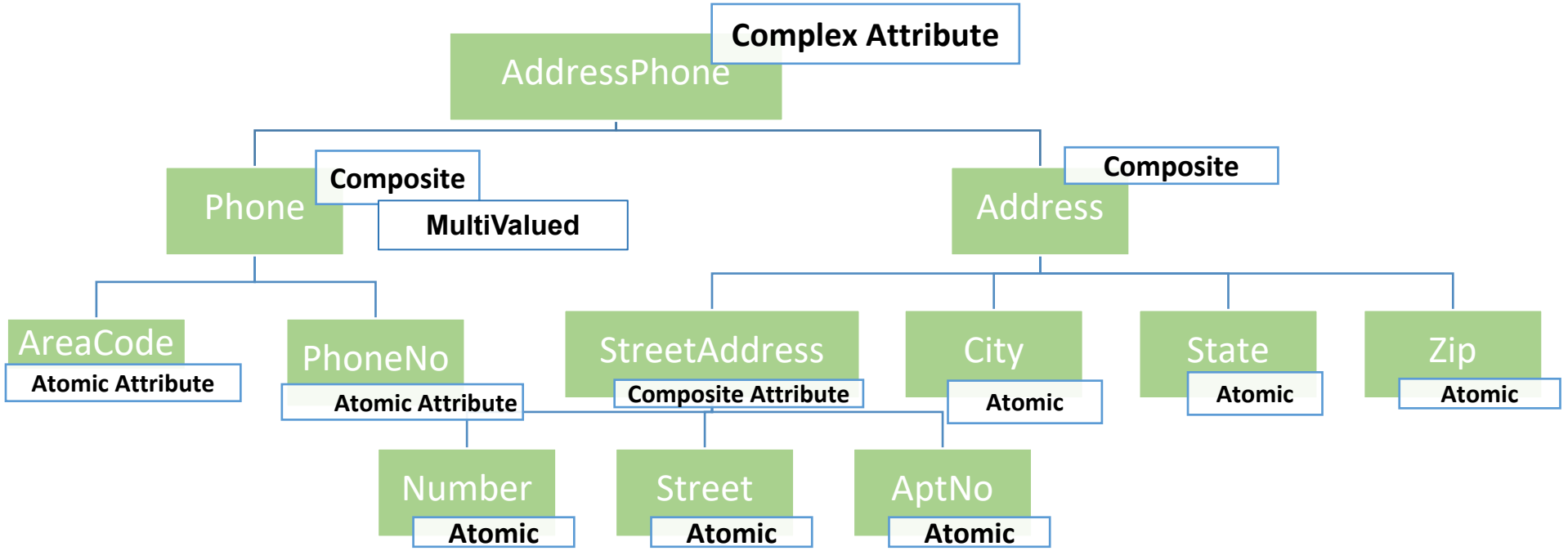
Complex Attributes

- ✓ A group of composite and multivalued attributes
- ✓ Composite attributes represented using ( ) where every atomic attribute separated by ,
- ✓ Multivalued attributes represented using { }
- ✓ **E.g.** AddressPhone for a person

  {AddressPhone({Phone (AreaCode, PhoneNo)} ,

  Address(StreetAddress(Number, Street,

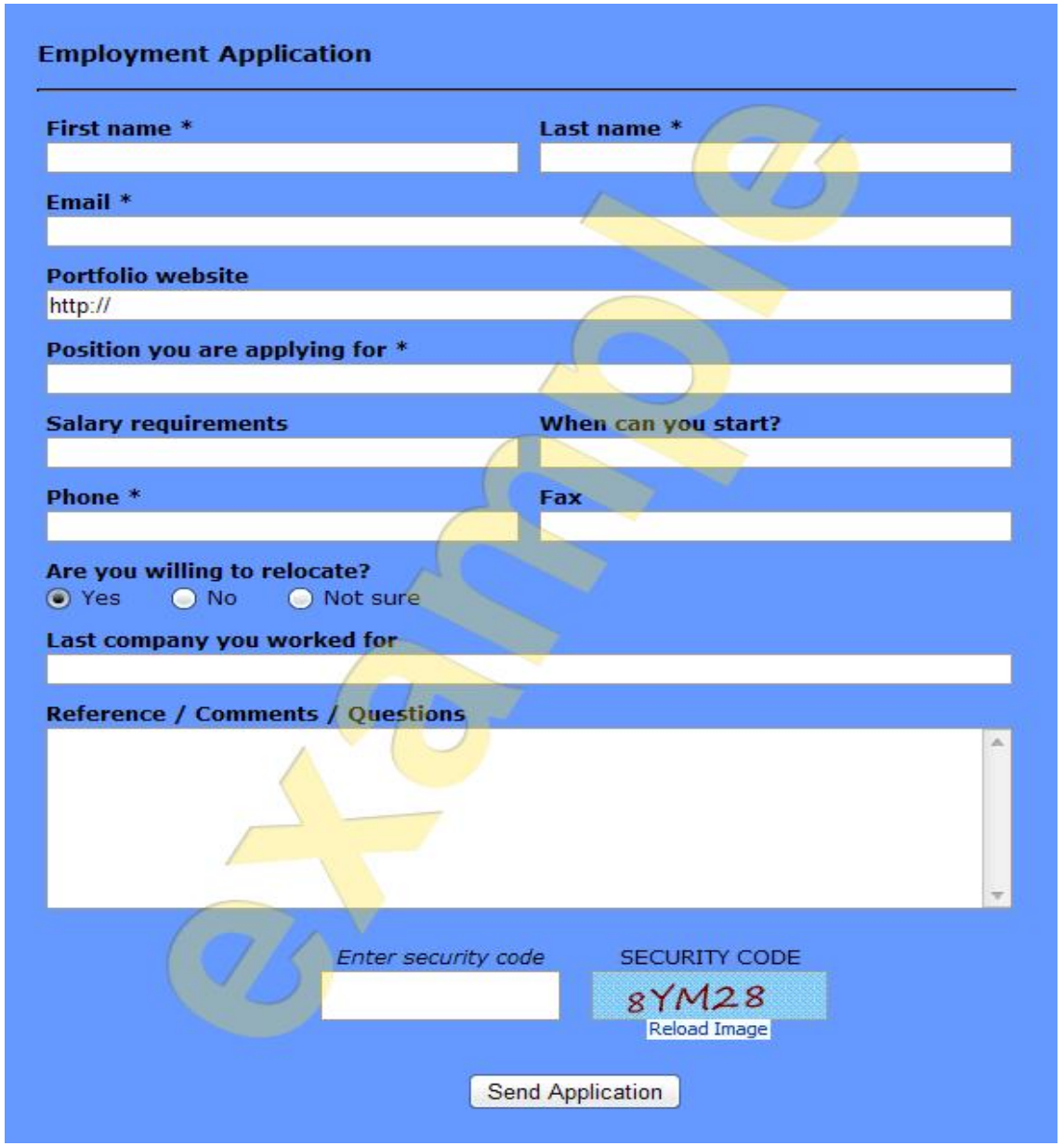
  AptNo), City, State, Zip)) }



{AddressPhone({Phone (AreaCode, PhoneNo)} ,

Address(StreetAddress(Number, Street, AptNo),

City, State, Zip)) }


Employment Application
First name *
Last name *
Email *
Portfolio website
http://
Position you are applying for *
Salary requirements
When can you start?
Phone *
Fax
Are you willing to relocate?
Yes
No
Not sure
Last company you worked for
Reference / Comments / Questions
Enter security code
SECURITY CODE
8YM28
Reload Image
Send Application
example

ER Diagram Notations

Entity

Attribute

Weak Entity

Key Attribute

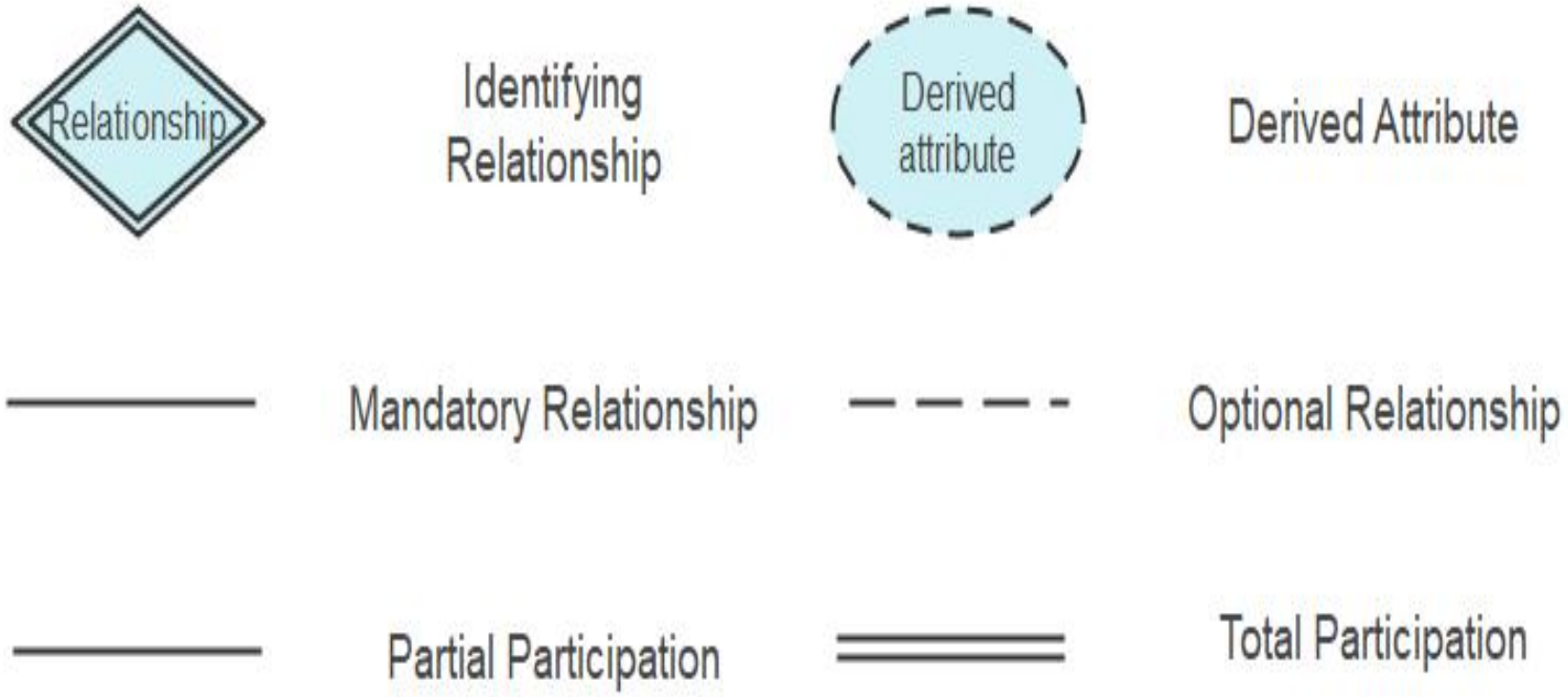
Associative Entity

Key Attribute

Relationship

Key Attribute

ER Diagram Notations (Continued)



ER Diagram Relationships

one-to-one (1:1)

one-to-many (1:N)

ER Diagram Relationships

many-to-one (N:1)

many-to-many (M:N)

Cardinality

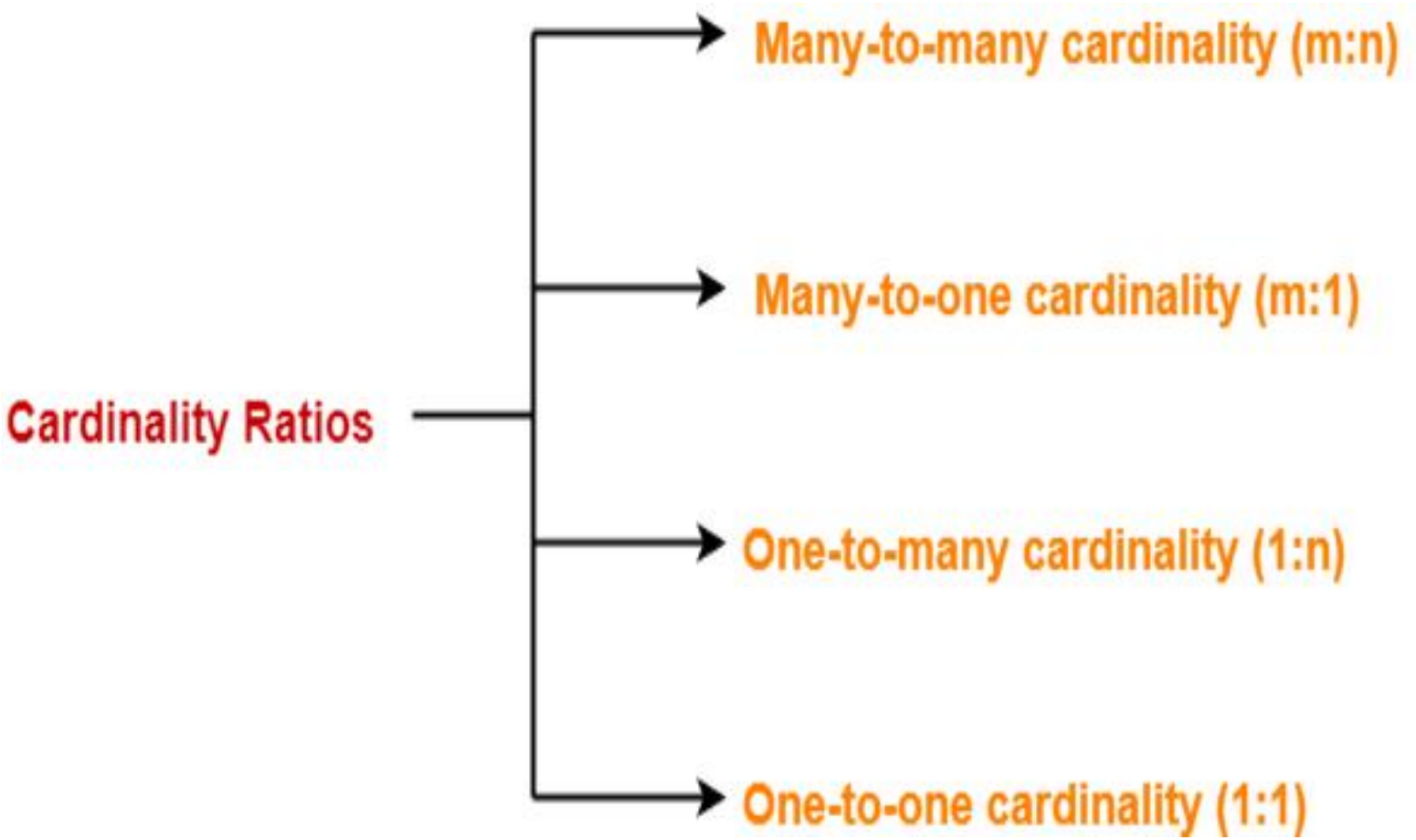
- Database administrators can apply cardinality to a database to describe how two different objects or entities relate to each other.

- What is Cardinality in DBMS:

- A number that represents, one instance of first entity(table) is related to how many instances of the second entity, is known as cardinality.

Cardinality Constraint-

- Types of Cardinality Ratios-
- There are 4 types of cardinality ratios-



1. Many-to-Many Cardinality-

- By this cardinality constraint,
- An entity in set A can be associated with any number (zero or more) of entities in set B.
- An entity in set B can be associated with any number (zero or more) of entities in set A.

1. Many-to-Many Cardinality- Symbol

Cardinality Ratio = m : n

1. Many-to-Many Cardinality- Example

- Here,

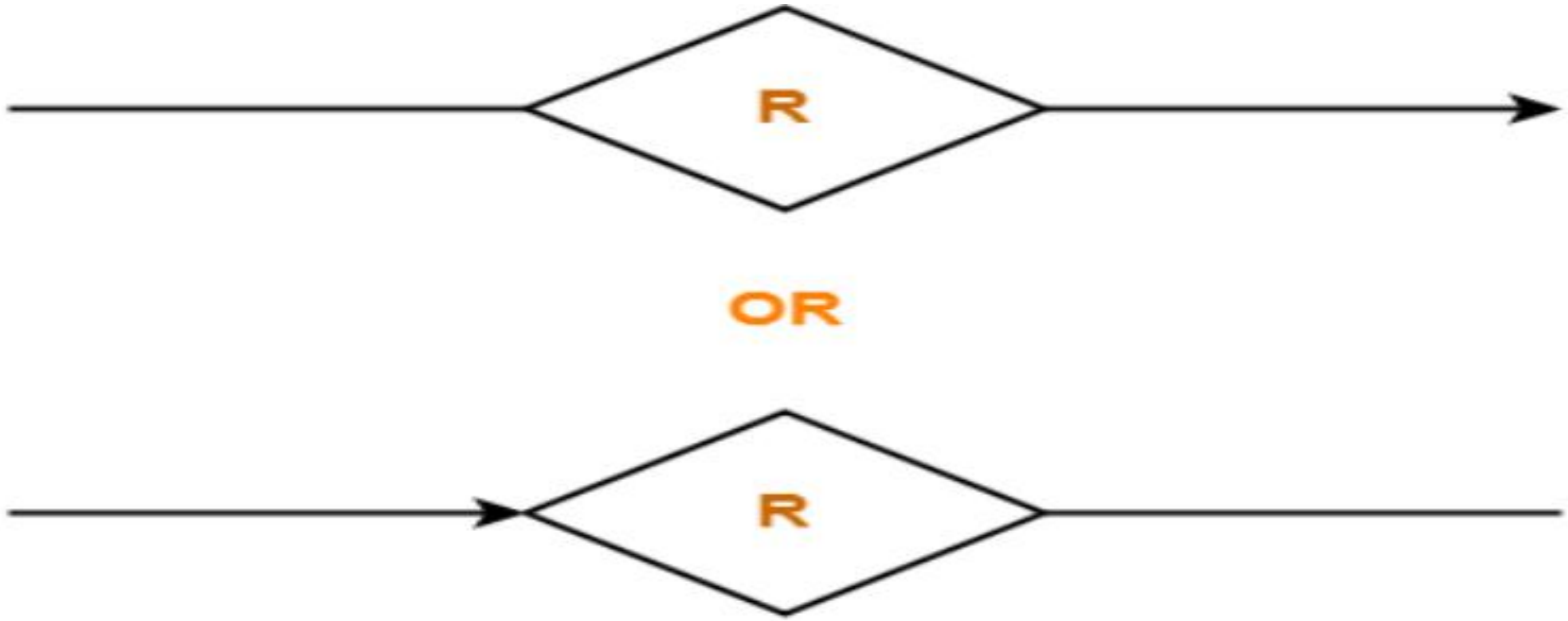
- One student can enroll in any number (zero or more) of courses.
- One course can be enrolled by any number (zero or more) of students.

2. Many-to-One Cardinality-

By this cardinality constraint,

- An entity in set A can be associated with at most one entity in set B.
- An entity in set B can be associated with any number (zero or more) of entities in set A.

2. Many-to-One Cardinality-Symbol



2. Many-to-One Cardinality-Example

Many to One Relationship

- Here,

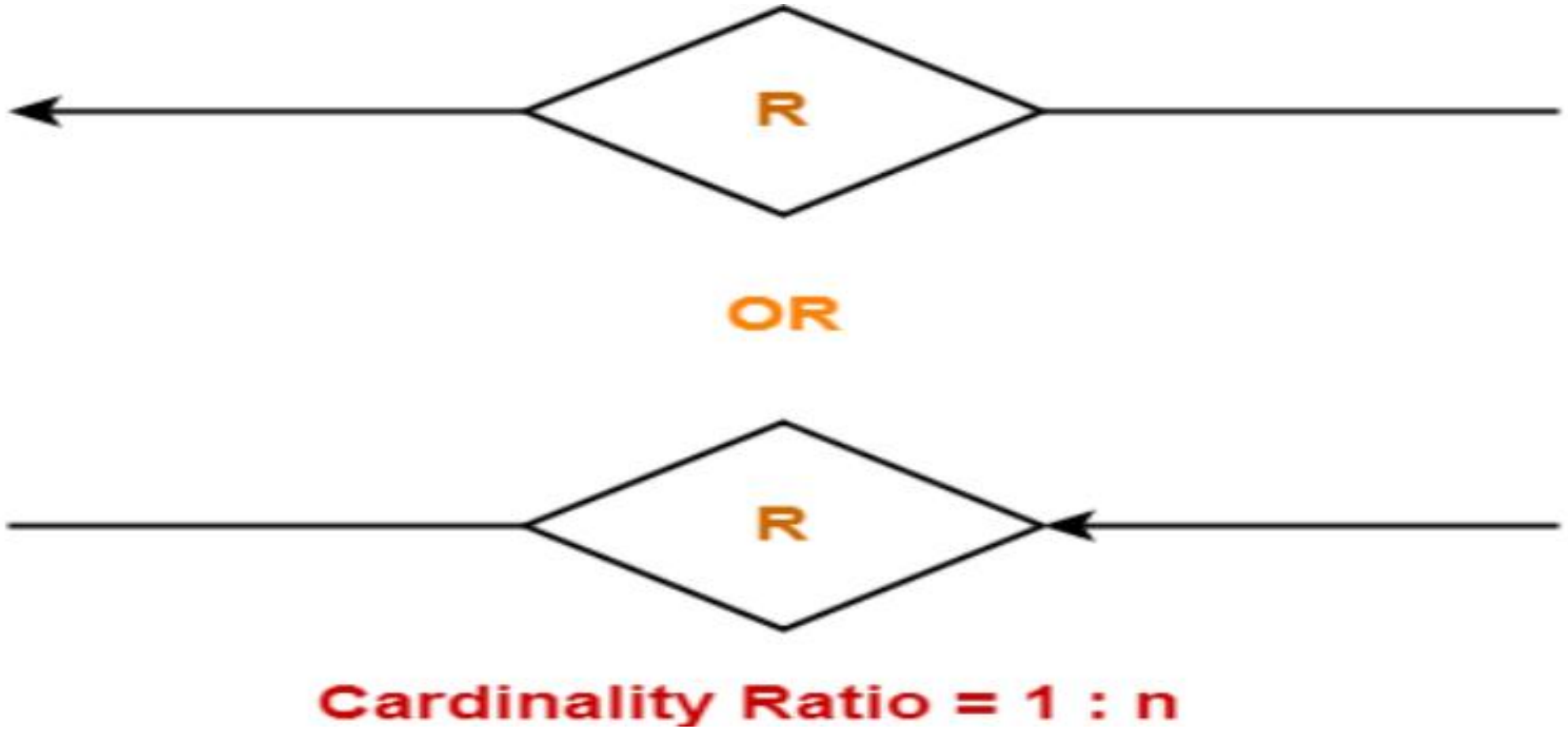
- One student can enroll in at most one course.
- One course can be enrolled by any number (zero or more) of students.

3. One-to-Many Cardinality-

- By this cardinality constraint,

- An entity in set A can be associated with any number (zero or more) of entities in set B.
- An entity in set B can be associated with at most one entity in set A.

3. One-to-Many Cardinality- Symbol



3. One-to-Many Cardinality- Example

One to Many Relationship

- Here,

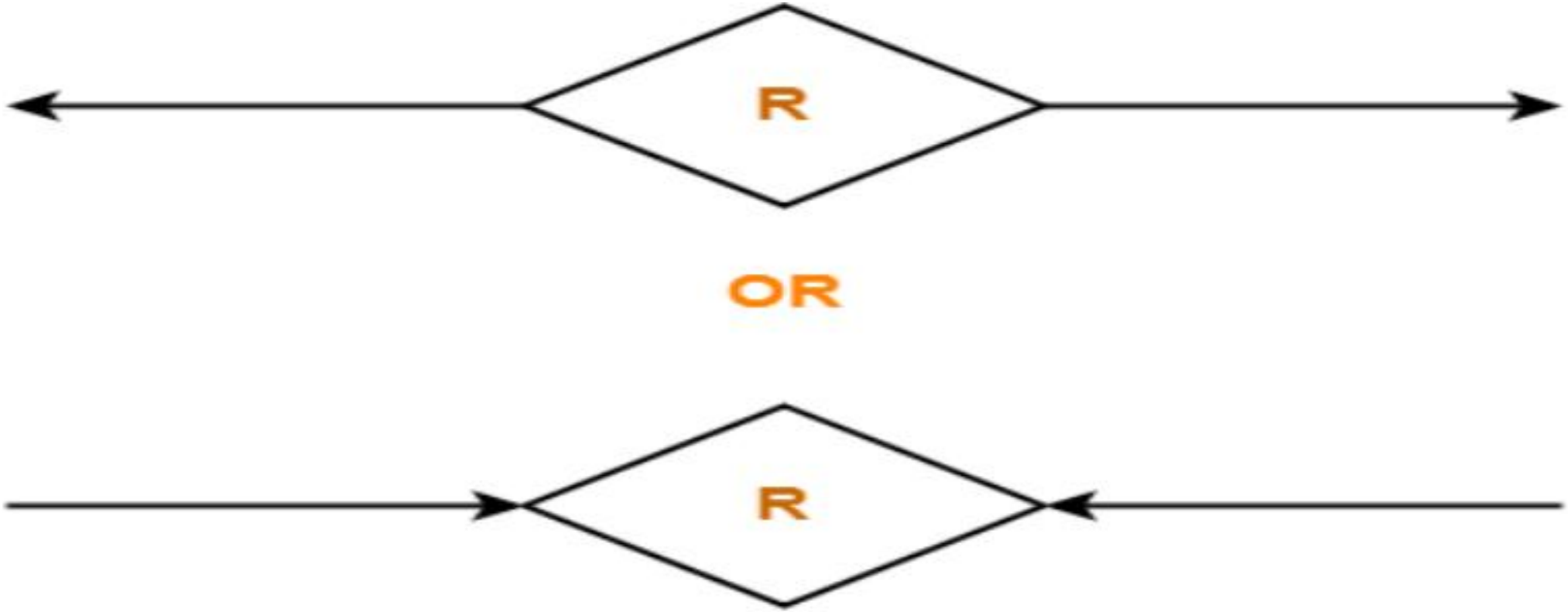
- One student can enroll in any number (zero or more) of courses.
- One course can be enrolled by at most one student.

4. One-to-One Cardinality-

By this cardinality constraint,

- An entity in set A can be associated with at most one entity in set B.
- An entity in set B can be associated with at most one entity in set A.
-

4. One-to-One Cardinality- Symbol



Cardinality Ratio = 1 : 1

What are Keys in DBMS?

One to One Relationship

- Here,

- One student can enroll in at most one course.
- One course can be enrolled by at most one student